CHAPTER 5

Modifying Busy Databases Without Downtime

Over time your application codebase and database structure will evolve. Part of your development team's velocity hinges on being able to have a short time to delivery for both new code features and incremental database modifications. Rails developers use Active Record Migrations to evolve the database structure, constantly creating DDL changes as needed for features and as a change management mechanism to keep all databases in sync.

When your application is new and usage is low, modifying the database structure incurs almost no risk. Tables that are locked have few rows and lock times are short. There may not be much concurrent activity from customers or internal users. As an application grows in popularity and usage, data growth increases, query volume increases, and your team may begin to feel more pressure from database changes that become riskier and reduce the team development velocity.

One of the main challenges you'll face in evolving your database structure is performing operations that cause table locks, and how to do those safely and quickly.

How do you identify these riskier types of changes and how can you prevent them?

Structural changes from Active Record Migration methods that create `ALTER TABLE`[1] SQL statements can be riskier for tables with high row counts and high query volume. Most structural changes lock the table in Exclusive Access

1. https://www.postgresql.org/docs/current/sql-altertable.html